

ChronoPath

Application de planification d'itinéraires sportifs

Documentation technique — Cours GIN, HEIG-VD — 2025–2026

1. Introduction et objectifs

ChronoPath est une application web de planification d'itinéraires sportifs développée dans le cadre du cours de Géoinformatique (GIN) à la HEIG-VD. Elle permet aux sportifs (cyclistes, randonneurs, coureurs) de planifier leurs sorties sur la Suisse romande en se basant sur des données cartographiques réelles et un calcul d'itinéraire sur le réseau routier OSM.

1.1 Objectifs pédagogiques

- Manipuler des données géospatiales réelles (OpenStreetMap, Swisstopo)
- Implémenter un moteur de routage sur graphe (pgRouting + Dijkstra)
- Développer une application full-stack avec un backend Go et un frontend Leaflet.js
- Intégrer des APIs cartographiques professionnelles (Swisstopo WMTS)
- Calculer des métriques sportives réalistes (FTP, VAP, Naismith)

1.2 Pourquoi ces choix technologiques ?

Chaque technologie a été choisie pour des raisons précises, détaillées dans les sections suivantes. Le principe directeur était d'utiliser des outils open-source professionnels, représentatifs du monde réel, tout en restant dans les capacités d'une équipe étudiante.

2. Architecture du projet

L'application suit une architecture client-serveur classique en trois couches :

Couche	
Frontend (client)	HTML/CSS/JS + Leaflet.js 1.9
Backend (serveur)	Go 1.21 + Gin framework
Base de données	PostgreSQL 15 + PostGIS + pgRouting 3.4
Données cartographiques	OSM + Swisstopo WMTS
Containerisation	Docker Desktop

2.1 Structure des fichiers

```
GIN_projet/
├── backend/
│   ├── main.go           # Point d'entrée, routes API
│   ├── .env              # Variables de connexion DB
│   └── handlers/
│       ├── routing.go    # Dijkstra, calcul itinéraires
│       ├── geodata.go    # Endpoint routes OSM
│       └── db.go         # Connexion PostgreSQL
├── frontend/
│   ├── index.html        # Interface principale
│   ├── css/style.css     # Styles
│   └── js/
│       ├── map.js        # Carte Leaflet + Swisstopo
│       ├── search.js     # Recherche pgRouting
│       ├── routing.js    # Itinéraire A→B + étapes
│       ├── create.js     # Créer + Sauvegardés
│       └── app.js        # Initialisation
└── Donnees_finales/
    └── CANTONS_Romands.gpkg # Données Swisstopo
```

3. Backend — Go + Gin

3.1 Pourquoi Go ?

Go (Golang) a été choisi pour le backend pour plusieurs raisons :

- Performances élevées : Go compile en binaire natif, les temps de réponse pour les calculs pgRouting sont très faibles (<5ms en moyenne)
- Typage statique : réduit les erreurs à la compilation, idéal pour un projet académique avec peu de tests
- Déploiement simple : un seul binaire exécutable, pas de dépendances runtime
- Concurrent natif : les goroutines permettent de gérer plusieurs requêtes simultanées sans effort

3.2 Pourquoi Gin ?

Gin est le framework web le plus populaire pour Go. Il offre :

- Router HTTP performant avec paramètres de chemin
- Middleware intégré (CORS, logging, recovery)
- Gestion des fichiers statiques (CSS, JS, images) via `r.Static()`
- API simple et proche de Express.js (familier pour les développeurs web)

3.3 Démarrage du serveur

À chaque session de travail, dans le terminal :

```
docker start gin_pgrounting
cd backend
go build -o server.exe . && server.exe
```

Le serveur démarre sur `http://localhost:8080`. Le message  PostgreSQL connecté confirme la connexion à la base de données.

 *go build recompile le binaire. Toujours relancer après une modification du code Go.*

4. Base de données — PostgreSQL + PostGIS + pgRouting

4.1 Pourquoi PostgreSQL ?

PostgreSQL est la base de données relationnelle open-source la plus avancée. Elle supporte nativement les extensions géospatiales nécessaires au projet, notamment PostGIS et pgRouting.

4.2 Pourquoi PostGIS ?

PostGIS ajoute le support des types géométriques (POINT, LINESTRING, POLYGON) et des fonctions spatiales (ST_Distance, ST_MakePoint, ST_Union, etc.) à PostgreSQL. C'est le standard de facto pour les bases de données géospatiales open-source.

- Stockage des géométries OSM en WGS84 (EPSG:4326)
- Calcul de distances et intersections directement en SQL
- Export en GeoJSON via ST_AsGeoJSON()

4.3 Pourquoi pgRouting ?

pgRouting est une extension PostgreSQL qui ajoute des algorithmes de routage sur graphe. Elle permet de calculer des itinéraires optimaux sur le réseau routier OSM importé.

- Algorithme Dijkstra : calcul du plus court chemin entre deux nœuds
- Données OSM importées via osm2pgrouting : 351 509 ways, 291 381 nœuds
- Coût basé sur cost_s (temps en secondes) pour des itinéraires réalistes

4.4 Import des données OSM

Les données OpenStreetMap sont importées via osm2pgrouting, qui crée automatiquement le graphe de routage avec les tables ways et ways_vertices_pgr.

```
osm2pgrouting --file switzerland.osm --conf mapconfig.xml
               --dbname webmapping --username gin --password ginpasse
```

4.5 Containerisation avec Docker

La base de données tourne dans un conteneur Docker pour garantir la reproductibilité de l'environnement entre les membres de l'équipe. La commande de création :

```
docker run --name gin_pgrouting -e POSTGRES_USER=gin
-e POSTGRES_PASSWORD=ginpass -e POSTGRES_DB=webmapping
-p 5432:5432 -d pgrouting/pgrouting:15-3.5-3.4.2
```

5. Algorithme de routage — Dijkstra

5.1 Principe

L'algorithme de Dijkstra calcule le chemin le plus court entre deux nœuds dans un graphe pondéré. pgRouting l'implémente sur le réseau routier OSM, où :

- Chaque way OSM est une arête du graphe
- Le coût est le temps de parcours (cost_s en secondes)
- La requête pgr_dijkstra retourne la séquence d'arêtes optimale

5.2 Requête SQL utilisée

```
SELECT ST_AsGeoJSON(ST_Union(w.the_geom)),
       COALESCE(SUM(w.length_m), 0)
FROM pgr_dijkstra(
  'SELECT gid AS id, source, target,
    cost_s AS cost, reverse_cost_s AS reverse_cost
   FROM ways WHERE cost_s > 0',
  $1::bigint, $2::bigint, true
) r JOIN ways w ON r.edge = w.gid WHERE r.edge > 0
```

5.3 Boucles triangulaires

Pour la fonctionnalité Rechercher, l'application génère des boucles en calculant 3 itinéraires dans des directions différentes (Nord-Est, Nord-Ouest, Sud-Est). Chaque boucle utilise 3 segments Dijkstra :

- Source → Point intermédiaire 1
- Point intermédiaire 1 → Point intermédiaire 2 (perpendiculaire)
- Point intermédiaire 2 → Source

Le rayon des points intermédiaires est calculé en fonction de la durée souhaitée et de la vitesse estimée, avec un facteur de sinuosité 2.6 (les routes ne sont jamais rectilignes).

💡 Fallback automatique en mode estimation directe si pgRouting ne trouve pas de chemin (zones sans réseau routier).

6. Frontend — Leaflet.js + Swisstopo

6.1 Pourquoi Leaflet.js ?

Leaflet.js est la bibliothèque de cartographie JavaScript open-source la plus utilisée. Elle offre :

- API simple et bien documentée, idéale pour un projet académique
- Support des tuiles WMTS (Swisstopo), GeoJSON, et couches personnalisées
- Plugins nombreux (Leaflet.draw pour le tracé manuel)
- Performance excellente pour afficher des milliers de géométries OSM

6.2 Fonds de carte Swisstopo

L'application utilise exclusivement les APIs Swisstopo (gratuites, sans clé) :

Fond de carte	
Carte Topographique	ch.swisstopo.pixelkarte-farbe — carte nationale suisse
Orthophoto	ch.swisstopo.swissimage — imagerie aérienne haute résolution

6.3 Structure des onglets

Onglet	
Rechercher	Génère 3 itinéraires pgRouting selon durée, sport et profil physiologique
Créer	Tracé manuel point par point avec calcul pgRouting en temps réel
Sauvegardés	Import GPX, visualisation, modification, export, calculateur de temps
Itinéraire	Calcul A→B avec étapes intermédiaires, chargement GPX

6.4 Géocodage

Le géocodage (adresse → coordonnées) utilise l'API Nominatim d'OpenStreetMap, gratuite et sans clé. Les résultats sont filtrés sur la Suisse (countrycodes=ch).

7. Calcul des métriques sportives

7.1 Pourquoi des métriques physiologiques ?

La durée d'un itinéraire dépend fortement du profil physique du sportif. Une durée de 2h pour un cycliste FTP 2 W/kg n'implique pas la même distance que pour un FTP 5 W/kg. L'application utilise des paramètres physiologiques standards pour estimer la vitesse et la durée.

7.2 Vélo — FTP (Functional Threshold Power)

La FTP (Puissance au Seuil Fonctionnel) est la puissance maximale qu'un cycliste peut maintenir pendant 1 heure, exprimée en W/kg. La formule utilisée :

```
vitesse (km/h) = 10 + FTP × 4
Débutant : × 0.75 | Moyen : × 1.0 | Expert : × 1.15
Limites : 12 - 45 km/h
```

Exemple : FTP 3.0 W/kg, niveau moyen → 22 km/h

7.3 Course — VAP (Vitesse Aérobie Maximale)

La VAP est la vitesse à laquelle le VO2max est atteint, exprimée en min/km. Les courses se courent à un pourcentage de la VAP selon la distance :

Distance	
≤ 5 km	
≤ 10 km	
≤ 21 km (semi)	
> 21 km (marathon+)	

Pénalité dénivelé : +0.5 min par 100m D+ (course sur route).

7.4 Rando — Formule de Naismith

La vitesse en randonnée est estimée selon le niveau : 3.0 / 4.0 / 5.5 km/h. La formule de Naismith ajoute 1 heure par 300m de dénivelé positif.

7.5 Extraction du dénivelé depuis les GPX

Lors de l'import d'un fichier GPX, le dénivelé positif est calculé automatiquement depuis les balises <ele> :

```
const eles = trkpts.map(pt => parseFloat(pt.querySelector('ele')?.textContent || 0));
```

```
for (i=1; i<eles.length; i++) { if (eles[i] - eles[i-1] > 0) deniv += eles[i] -  
eles[i-1]; }
```

8. Données utilisées

Source	
OpenStreetMap	
Swisstopo WMTS	
CANTONS_Romands.gpkg	
Nominatim (OSM)	
Fichiers GPX utilisateur	

8.1 Couche Routes OSM

L'endpoint `/api/osm-ways` retourne les routes OSM depuis la base de données pgRouting, filtrées par bounding box et limitées à 2000 éléments. La couleur d'affichage dépend du tag_id OSM :

- Rouge : autoroutes (tag_id 101-102)
- Orange : routes principales (tag_id 106-107)
- Jaune : routes secondaires (tag_id 108-109)
- Vert : pistes cyclables (tag_id 118)
- Bleu : autres routes

9. Endpoints API

Endpoint	
GET /api/search-routes	Génère 3 itinéraires pgRouting selon profil sportif
GET /api/route	Calcul itinéraire A→B (Dijkstra)
GET /api/osm-ways	Routes OSM dans un bounding box
GET /api/health	Statut de la connexion DB
GET /api/pois	Points d'intérêt

Paramètres /api/search-routes

```
fromLon, fromLat : coordonnées WGS84 du point de départ
sport             : velo | rando | course
mode             : boucle | destination
duree_h          : durée souhaitée en heures
niveau           : debutant | moyen | expert
ftp              : W/kg (vélo uniquement)
vap              : min/km (course uniquement)
```

10. Guide de démarrage

10.1 Prérequis (installation unique)

- Go 1.21+ : <https://go.dev/dl/>
- Docker Desktop : <https://www.docker.com/products/docker-desktop/>

10.2 À chaque session

1. Démarrer Docker Desktop

2. Dans un terminal :

```
docker start gin_pgrounting
cd C:\...\GIN_projet\backend
go build -o server.exe . && server.exe
```

3. Ouvrir <http://localhost:8080> dans le navigateur

4. Forcer le rechargement si nécessaire : Ctrl+Shift+R

10.3 Dépannage

Problème	
PostgreSQL non connecté	Vérifier que Docker est démarré : <code>docker start gin_pgrounting</code>
ERR_CONNECTION_REFUSED	Relancer <code>go build -o server.exe . && server.exe</code>
Itinéraires en estimation	La DB n'est pas connectée, vérifier <code>.env</code> et Docker
Routes OSM absentes	Dézoomer, limite 2000 éléments par vue
Modifications JS ignorées	Ctrl+Shift+R pour vider le cache navigateur
go build : cannot find main module	Vérifier d'être dans le dossier backend/

11. Fonctionnalités implémentées

Onglet Rechercher

- Saisie du point de départ par adresse, GPS ou clic carte
- Mode Boucle : 3 boucles triangulaires pgRouting (Nord-Est, Nord-Ouest, Sud-Est)
- Mode Destination : 3 itinéraires one-way dans différentes directions
- Filtres selon le sport : FTP (vélo), VAP (course), durée, niveau, dénivelé
- Export GPX de chaque itinéraire

Onglet Créer

- Tracé manuel point par point sur la carte
- Calcul pgRouting en temps réel entre chaque segment
- Affichage distance et durée estimée après calcul
- Sauvegarde dans les Sauvegardés

Onglet Sauvegardés

- Import de fichiers GPX avec extraction automatique du dénivelé
- Visualisation sur la carte
- Modification : rechargement des points dans l'onglet Créer
- Calculateur de temps : VAP, FTP, niveau → durée estimée
- Export GPX

Onglet Itinéraire

- Calcul A→B avec étapes intermédiaires multiples
- Chargement et affichage de fichiers GPX
- Export GPX du résultat

Couches cartographiques

- Carte Topographique Swisstopo
- Orthophoto Swisstopo
- Cantons romands (GeoPackage Swisstopo)
- Routes OSM dynamiques avec légende (rechargement automatique au zoom)